TUTORIAL DE GIT

Capítulo 4. Git en el servidor

Juan Felipe Castañeda Rodríguez · Fundación Universitaria Agraria de Colombia (UNIAGRARIA)

Ciencia de Datos — Actividad 2: Hands on, configuración de herramientas · Octubre de 2026

Para colaborar se necesita un repositorio remoto al que todos tengan acceso. Este capítulo explica los protocolos de comunicación y cómo montar un servidor Git propio, además de las alternativas alojadas.

4.1 Los protocolos

Git puede transferir datos usando cuatro protocolos.

Local. El remoto es otra carpeta en el mismo disco o en una unidad compartida.

```
git clone /srv/git/proyecto.git
git clone file:///srv/git/proyecto.git
```

Es simple, pero depende del acceso a la carpeta compartida.

HTTP(S). El llamado "HTTP inteligente" permite clonar y empujar con usuario y contraseña (o token) sobre el puerto 443. Es el más usado en GitHub porque atraviesa casi cualquier firewall.

```
git clone https://github.com/usuario/proyecto.git
```

SSH. Muy común en servidores propios; es seguro, eficiente y viene instalado en la mayoría de sistemas. No permite acceso anónimo.

```
git clone ssh://usuario@servidor/proyecto.git
git clone usuario@servidor:proyecto.git
```

Git. Un demonio especial en el puerto 9418; es el más rápido pero no tiene autenticación, así que solo sirve para acceso de lectura público.

4.2 Configurar Git en un servidor

Un servidor necesita un **repositorio desnudo** (bare): sin directorio de trabajo, solo con el contenido de .git.

```
git clone --bare mi_proyecto mi_proyecto.git
scp -r mi_proyecto.git usuario@servidor:/srv/git
```

Cualquier persona con acceso SSH a esa ruta ya puede clonar y empujar. Para crear uno vacío directamente en el servidor:

```
git init --bare --shared
```

--shared ajusta los permisos para que varios usuarios del mismo grupo puedan escribir.

4.3 Generar una clave SSH

Las claves SSH evitan escribir contraseñas en cada operación y son el método recomendado también para GitHub.

```
ssh-keygen -t ed25519 -C "tu_correo@ejemplo.com"
```

```
cat ~/.ssh/id_ed25519.pub
```

La clave pública (.pub) se entrega al servidor o se pega en la configuración de la cuenta de GitHub; la privada nunca se comparte.

4.4 Configurar el servidor

En el servidor se crea un usuario git, se agrega cada clave pública al archivo ~/.ssh/authorized_keys y se crean los repositorios desnudos. Para que los usuarios solo puedan usar Git y no una terminal completa, se puede asignar git-shell como intérprete del usuario git.

4.5 Demonio Git, HTTP inteligente y GitWeb

- **git daemon** sirve repositorios públicos de solo lectura.
- **HTTP inteligente** se configura con el CGI git-http-backend sobre Apache o Nginx.
- **GitWeb** es una interfaz web sencilla para navegar repositorios:

```
git instaweb --httpd=webrick
```

4.6 GitLab

GitLab es una aplicación web completa y de código abierto para alojar repositorios en un servidor propio. Ofrece usuarios, grupos, permisos, solicitudes de fusión (merge requests), integración continua y gestión de incidencias, todo administrado desde el navegador.

4.7 Alojamiento de terceros

Si no se quiere mantener un servidor, existen servicios alojados como GitHub, GitLab.com o Bitbucket. Se crea una cuenta, se crea el repositorio y se trabaja inmediatamente. Para esta actividad se usa GitHub, que se estudia en detalle en el capítulo 6.

4.8 Resumen

Para colaborar hace falta un remoto. Se puede montar uno propio con un repositorio desnudo y SSH, o usar un servicio alojado. HTTPS y SSH son los protocolos más usados; las claves SSH son la forma más cómoda y segura de autenticarse.

Referencia

Chacon, S., & Straub, B. (2014). *Pro Git* (2.ª ed.). Apress. https://git-scm.com/book/es/v2